

University of Tehran
College of Engineering
School of Electrical and
Computer Engineering

Machine Learning

Assignment 3

Name

Parsa KafshduziBukani

Student ID

810102501

December 9, 2025

Contents

Table of Contents	2
List of Figures	3
List of Tables	3
Abstract	4
Question 1 — Support Vector Machines: Conceptual Questions	5
1. Why is the dual formulation of SVM usually preferred when kernels are used?	5
2. Why do only support vectors influence the final SVM classifier?	5
3. Why the kernel trick allows operation in infinite-dimensional spaces	6
4. Effect of kernel choice on decision boundary, capacity, and overfitting	6
Question 2 — RBF Kernel Feature Space Intuition	7
Question 3 — SVM Dual Symmetric 4-Point Example	8
Question 4 — Explicit Cubic Feature Map for a Polynomial Kernel	10
1. Expansion using the multinomial theorem	10
2. Construction of an explicit feature map	11
3. Verification that $K(x, z) = \phi(x)^\top \phi(z)$	11
Question 5 — The Mechanics of Splitting (ID3)	12
1. Root Entropy	12
2. Information Gain for X_1 and X_2	12
3. Full decision tree and training error	13
4. Top-down vs. Bottom-up tree construction	14
Question 6 — Optimization Geometry & Jensen's Inequality	14
1. Concavity of the Entropy Function	14
2. Jensen's Inequality and Non-Negativity of Information Gain	15
3. Optimization Trap: Hill-Climbing Failure in ID3	16

Question 7 — The Failure of Greedy Search (XOR)	16
1. The Greedy Trap: Information Gain for X_1 and X_2	16
2. A Depth-2 Tree that Solves XOR	17
3. Inductive Bias: Preference vs. Restriction	18
Question 8 — Diabetes Classification Using Support Vector Machines	18
Part 1: Data Preprocessing and Exploration	18
Part 2: Linear SVM Classification	19
Part 3: Comparison of Linear, RBF, and Polynomial Kernels . .	19
Part 4: Hyperparameter Tuning	20
Part 5: PCA and 2D Visualization	20
Part 6: Class Imbalance Analysis	22
Question 9 — Primal Soft-Margin SVM via Quadratic Programming	22
Part 1: Data Preprocessing	22
Part 2: Solving the Primal SVM Using Quadratic Programming	22
Part 3: Model Evaluation	23
Part 4: Decision Boundary Visualization	23
Question 10 — Decision Tree Classifier (ID3) from Scratch	23
Part 1: Data Loading and Pre-processing	23
Part 2: Entropy and Information Gain Computation	24
Part 3: Handling Continuous Features	25
Part 4: Decision Tree Construction and Evaluation	25
Part 5: Overfitting, Noise Sensitivity, and Pruning	25

List of Figures

1	Confusion matrices for Linear SVM: unscaled (left) and scaled (right).	19
2	Confusion matrices for Linear, RBF, and Polynomial SVMs.	20
3	Training set after PCA reduction to 2D.	21
4	RBF SVM decision boundary in the PCA-reduced space.	21
5	QP-based soft-margin SVM decision boundary using optimal parameters w^* and b^*	24
6	Tree Size vs. Test Accuracy during Reduced Error Pruning.	26

List of Tables

Abstract

This assignment explored several core algorithms in machine learning, including linear optimization methods, support vector machines, kernel-based classification, and decision tree learning. Theoretical exercises examined the geometry of line search methods, the role of descent directions, and the behavior of step-size rules such as exact search and backtracking. We also analyzed the dual formulation of SVMs, kernel-induced feature spaces, and the structural properties of maximum-margin classifiers, including the significance of support vectors and the implications of various kernel choices on model capacity and overfitting.

A major component of the assignment involved hands-on implementation and experimentation. Using the Pima Indians Diabetes dataset, we constructed a full SVM classification pipeline, including preprocessing, feature scaling, kernel comparison, hyperparameter tuning, PCA visualization, and class-imbalance analysis. The results demonstrated the sensitivity of SVMs to normalization, the superior expressiveness of nonlinear kernels, and the effects of class-weighting on minority-class recall.

We further implemented the primal soft-margin SVM directly as a Quadratic Program using `cvxpy`, recovering the optimal separating hyperplane without relying on built-in SVM libraries. Visualization of the learned decision boundary confirmed correct margin-maximizing behavior.

Finally, we developed a complete ID3 decision tree classifier from scratch, supporting both discrete and continuous attributes through entropy-based splitting and threshold optimization. Experiments on the Extended Jeeves “Play Tennis” dataset highlighted overfitting tendencies, sensitivity to label noise, and the influence of pruning on generalization. A pruning analysis demonstrated the classical trade-off between tree complexity and predictive performance, reinforcing theoretical expectations about bias-variance behavior in decision trees.

Across all tasks, the assignment integrated theoretical understanding with practical modeling, illustrating how optimization principles, geometric intuition, and algorithmic design jointly shape the performance of foundational machine learning methods.

Question 1 — Support Vector Machines: Conceptual Questions

1. Why is the dual formulation of SVM usually preferred when kernels are used?

The dual formulation rewrites the optimization problem entirely in terms of inner products $x_i^\top x_j$. When using kernels, this form becomes advantageous for two main reasons:

- **Kernel substitution becomes possible.** In the dual, every data interaction appears as $x_i^\top x_j$. Replacing these with a kernel $K(x_i, x_j)$ allows computation in high- or infinite-dimensional spaces without explicitly constructing the feature map.
- **Dependence only on support vectors.** The dual solution assigns nonzero coefficients only to support vectors, yielding computational and memory efficiency.

2. Why do only support vectors influence the final SVM classifier?

The SVM decision function is:

$$f(x) = \sum_{i=1}^n \alpha_i y_i K(x_i, x) + b.$$

From the KKT conditions:

$$\alpha_i = 0 \quad \text{whenever} \quad y_i(w^\top x_i + b) > 1.$$

Thus, points that are correctly classified and lie strictly outside the margin have:

$$\alpha_i = 0.$$

Only points exactly on the margin—or violating it—receive $\alpha_i > 0$ and thus contribute to $f(x)$. These are the ****support vectors****, and they alone define the final hyperplane.

3. Why the kernel trick allows operation in infinite-dimensional spaces

Let $\phi(x)$ be a feature map (possibly infinite-dimensional). The dual only needs inner products:

$$\phi(x_i)^\top \phi(x_j).$$

If we define:

$$K(x_i, x_j) = \phi(x_i)^\top \phi(x_j),$$

then:

- We never need to compute $\phi(x)$ explicitly.
- All computations occur using $K(x_i, x_j)$, which is cheap even if $\phi(x)$ is infinite-dimensional.
- SVM optimization remains finite-dimensional and tractable.

Thus, SVMs effectively operate in extremely high-dimensional feature spaces while performing only kernel evaluations.

4. Effect of kernel choice on decision boundary, capacity, and overfitting

- **Linear Kernel** Produces a linear separating hyperplane.
 - Low model complexity.
 - Works well when classes are nearly linearly separable.
 - Low risk of overfitting.
- **Polynomial Kernel** Produces polynomial decision surfaces.
 - Capacity increases with degree d .
 - Higher degrees can fit complex patterns but risk overfitting.
 - Good for interactions between features.
- **RBF (Gaussian) Kernel** Generates very flexible nonlinear boundaries.
 - Infinite-dimensional feature space.
 - Controlled by γ (or σ):

- * Large γ : highly complex boundary $\rightarrow$ high overfitting risk.
- * Small γ : smoother boundary $\rightarrow$ better generalization.
- Often performs best when data is not linearly separable.

Question 2 — RBF Kernel Feature Space Intuition

The Gaussian (RBF) kernel is defined as:

$$K(x, z) = \exp \left(-\frac{\|x - z\|^2}{2\sigma^2} \right).$$

To understand why this kernel corresponds to an **inner product in an infinite-dimensional Hilbert space**, we expand and rewrite the expression mathematically.

1. Expansion of the kernel Rewrite the squared norm:

$$\|x - z\|^2 = \|x\|^2 + \|z\|^2 - 2x^\top z.$$

Thus,

$$K(x, z) = \exp \left(-\frac{\|x\|^2}{2\sigma^2} \right) \exp \left(-\frac{\|z\|^2}{2\sigma^2} \right) \exp \left(\frac{x^\top z}{\sigma^2} \right).$$

Only the final exponential term contains the interaction between x and z .

2. Infinite polynomial expansion Use the Taylor expansion:

$$\exp(t) = \sum_{k=0}^{\infty} \frac{t^k}{k!}.$$

Apply this to the interaction term:

$$\exp \left(\frac{x^\top z}{\sigma^2} \right) = \sum_{k=0}^{\infty} \frac{1}{k!} \left(\frac{x^\top z}{\sigma^2} \right)^k.$$

Each power $(x^\top z)^k$ expands into a sum of products of degree- k monomials of x and z . Thus, each term represents a dot product between vectors of monomials.

3. Implicit infinite-dimensional feature map Define the feature map:

$$\phi(x) = \exp\left(-\frac{\|x\|^2}{2\sigma^2}\right) \left[1, \frac{x}{\sigma}, \frac{1}{\sqrt{2!}} \left(\frac{x \otimes x}{\sigma^2}\right), \frac{1}{\sqrt{3!}} \left(\frac{x^{\otimes 3}}{\sigma^3}\right), \dots\right].$$

This vector contains:

- all monomials of degree 0 (a constant),
- all monomials of degree 1,
- all monomials of degree 2,
- ...

Therefore, $\phi(x)$ has **countably infinite dimension**.

4. Final interpretation By construction:

$$K(x, z) = \phi(x)^\top \phi(z),$$

but $\phi(x)$ is infinite-dimensional.

Thus, the RBF kernel corresponds to an **inner product in an infinite-dimensional Hilbert space**, even though we never compute the infinite vector — this is exactly the power of the kernel trick.

Question 3 — SVM Dual Symmetric 4-Point Example

We are given four points arranged symmetrically in $\mathbb{R}^2$:

$$\begin{aligned} x_1 &= (1, 1)^\top, \quad y_1 = +1, & x_2 &= (1, -1)^\top, \quad y_2 = +1, \\ x_3 &= (-1, 1)^\top, \quad y_3 = -1, & x_4 &= (-1, -1)^\top, \quad y_4 = -1. \end{aligned}$$

We use a hard-margin SVM with a linear kernel. The dual optimization problem is:

$$\max_{\alpha \geq 0} \sum_{i=1}^4 \alpha_i - \frac{1}{2} \sum_{i=1}^4 \sum_{j=1}^4 \alpha_i \alpha_j y_i y_j x_i^\top x_j,$$

subject to the equality constraint:

$$\sum_{i=1}^4 \alpha_i y_i = 0.$$

1. Computing the Gram matrix The dot products are:

$$x_1^\top x_1 = 2, \quad x_2^\top x_2 = 2, \quad x_3^\top x_3 = 2, \quad x_4^\top x_4 = 2,$$

and for cross terms:

$$\begin{aligned} x_1^\top x_2 &= 0, & x_1^\top x_3 &= 0, & x_1^\top x_4 &= -2, \\ x_2^\top x_3 &= -2, & x_2^\top x_4 &= 0, & x_3^\top x_4 &= 0. \end{aligned}$$

2. Symmetry argument The dataset is perfectly symmetric:

- Both positive-class points have the same norm and symmetric coordinates. - Both negative-class points are symmetric reflections of the positives.

Thus, by symmetry:

$$\alpha_1 = \alpha_2 = a, \quad \alpha_3 = \alpha_4 = b.$$

The constraint $\sum \alpha_i y_i = 0$ becomes:

$$a + a - b - b = 0 \Rightarrow 2a = 2b \Rightarrow a = b.$$

Let each dual variable be:

$$\alpha_1 = \alpha_2 = \alpha_3 = \alpha_4 = a.$$

3. Plugging into the dual objective The dual objective becomes:

$$L(a) = 4a - \frac{1}{2} \left(\sum_{i,j} a^2 y_i y_j x_i^\top x_j \right).$$

Compute the sum:

$$\sum_{i,j} y_i y_j x_i^\top x_j = 2 + 2 + 2 + 2 + 4 + 4 = 16.$$

Thus:

$$L(a) = 4a - \frac{1}{2}(a^2)(16) = 4a - 8a^2.$$

Maximize by setting derivative to zero:

$$\frac{dL}{da} = 4 - 16a = 0 \Rightarrow a = \frac{1}{4}.$$

Thus, the optimal dual variables are:

$$\alpha_1 = \alpha_2 = \alpha_3 = \alpha_4 = \frac{1}{4}.$$

4. Compute the primal parameters

Weight vector

$$w = \sum_{i=1}^4 \alpha_i y_i x_i.$$

Plug in values:

$$w = \frac{1}{4} \left[(+1)(1, 1) + (+1)(1, -1) + (-1)(-1, 1) + (-1)(-1, -1) \right].$$

Compute inside the brackets:

$$(1, 1) + (1, -1) + (-1, 1) + (-1, -1) = (4, 0).$$

Thus:

$$w = \frac{1}{4}(4, 0) = (1, 0).$$

Bias term Support vectors satisfy:

$$y_i(w^\top x_i + b) = 1.$$

Use $x_1 = (1, 1)$:

$$1(1 \cdot 1 + 0 \cdot 1 + b) = 1 \quad \Rightarrow \quad 1 + b = 1 \quad \Rightarrow \quad b = 0.$$

Final classifier

$$w = (1, 0), \quad b = 0.$$

Thus, the decision boundary is:

$$x_1 = 0,$$

which perfectly separates the positive points (on the right) from the negative points (on the left).

Question 4 — Explicit Cubic Feature Map for a Polynomial Kernel

1. Expansion using the multinomial theorem

We consider the cubic kernel:

$$K(x, z) = (1 + x^\top z)^3 = (1 + x_1 z_1 + x_2 z_2)^3,$$

where $x = (x_1, x_2)^\top$ and $z = (z_1, z_2)^\top$.

Using the multinomial theorem for $(a + b + c)^3$:

$$(a + b + c)^3 = a^3 + b^3 + c^3 + 3a^2b + 3a^2c + 3b^2a + 3c^2a + 6abc.$$

Let

$$a = 1, \quad b = x_1z_1, \quad c = x_2z_2.$$

Thus the expansion becomes:

$$K(x, z) = 1 + 3x_1z_1 + 3x_2z_2 + 3x_1^2z_1^2 + 6x_1x_2z_1z_2 + 3x_2^2z_2^2 + x_1^3z_1^3 + x_2^3z_2^3.$$

The number of monomials of total degree ≤ 3 in 2 variables is:

$$\binom{3+2}{3} = 10.$$

Thus, the resulting feature space is **10-dimensional**.

2. Construction of an explicit feature map

We seek a map $\phi : \mathbb{R}^2 \rightarrow \mathbb{R}^{10}$ such that:

$$K(x, z) = \phi(x)^\top \phi(z).$$

A valid choice with correct square-root scaling factors is:

$$\phi(x) = \left(1, \sqrt{3}x_1, \sqrt{3}x_2, \sqrt{3}x_1^2, \sqrt{6}x_1x_2, \sqrt{3}x_2^2, x_1^3, \sqrt{3}x_1^2x_2, \sqrt{3}x_1x_2^2, x_2^3\right)^\top.$$

Each coefficient (e.g., $\sqrt{3}$, $\sqrt{6}$) is chosen so that the dot product $\phi(x)^\top \phi(z)$ produces the exact expanded polynomial coefficients from part 1.

3. Verification that $K(x, z) = \phi(x)^\top \phi(z)$

We compute the dot product term by term:

$$\begin{aligned} \phi(x)^\top \phi(z) &= 1 + 3x_1z_1 + 3x_2z_2 + 3x_1^2z_1^2 + 6x_1x_2z_1z_2 + 3x_2^2z_2^2 \\ &\quad + x_1^3z_1^3 + 3x_1^2x_2z_1^2z_2 + 3x_1x_2^2z_1z_2^2 + x_2^3z_2^3. \end{aligned}$$

This exactly matches the multinomial expansion of:

$$(1 + x_1z_1 + x_2z_2)^3,$$

so indeed:

$$K(x, z) = \phi(x)^\top \phi(z).$$

Question 5 — The Mechanics of Splitting (ID3)

1. Root Entropy

The dataset contains 6 samples:

$$Y = \{0, 0, 1, 0, 1, 1\}.$$

Count labels:

$$n_0 = 3, \quad n_1 = 3, \quad n = 6.$$

Thus the class probabilities are:

$$p_0 = \frac{3}{6} = \frac{1}{2}, \quad p_1 = \frac{3}{6} = \frac{1}{2}.$$

The entropy is:

$$H(Y) = -p_0 \log_2 p_0 - p_1 \log_2 p_1 = -\frac{1}{2} \log_2 \frac{1}{2} - \frac{1}{2} \log_2 \frac{1}{2} = 1.$$

So the dataset has entropy:

$$\boxed{H(Y) = 1 \text{ bit}}.$$

2. Information Gain for X_1 and X_2

Gain for X_1 : Values of X_1 : A or B.

X_1	Y	Samples
A	$\{0, 0, 1\}$	(1, 2, 3)
B	$\{0, 1, 1\}$	(4, 5, 6)

Each subset contains 3 samples: 2 of one class, 1 of the other.

Thus each side has entropy:

$$H = -\frac{2}{3} \log_2 \frac{2}{3} - \frac{1}{3} \log_2 \frac{1}{3} = 0.9183.$$

Weighted entropy:

$$H(Y|X_1) = \frac{3}{6}(0.9183) + \frac{3}{6}(0.9183) = 0.9183.$$

Information gain:

$$\text{Gain}(S, X_1) = 1 - 0.9183 = 0.0817.$$

Gain for X_2 : Values of X_2 : C or D.

X_2	Y	Samples
C	$\{0, 0, 0\}$	$(1, 2, 4)$
D	$\{1, 1, 1\}$	$(3, 5, 6)$

Both subsets are pure:

$$H(Y|X_2=C) = 0, \quad H(Y|X_2=D) = 0.$$

Weighted entropy:

$$H(Y|X_2) = 0.$$

Thus:

$$\text{Gain}(S, X_2) = 1.$$

ID3 root choice ID3 selects the feature with maximum Information Gain:

$$\text{ID3 chooses } X_2 \text{ as the root node.}$$

3. Full decision tree and training error

From part 2:

- If $X_2 = C$, all labels are 0. - If $X_2 = D$, all labels are 1.

Thus the tree is:

$$\text{Root: } X_2 \rightarrow \begin{cases} C \Rightarrow Y = 0, \\ D \Rightarrow Y = 1. \end{cases}$$

This tree perfectly classifies all 6 samples.

$$\text{Training error} = 0.$$

No further splits are needed because both child nodes are pure.

4. Top-down vs. Bottom-up tree construction

ID3 works by:

- selecting the best split at the root, - then recursively splitting subsets. This produces a **top-down greedy tree**.

If we instead used a **bottom-up** or **post-hoc merging** approach, the tree:

- would not necessarily choose the same root, - might compress or merge nodes differently, - may produce a tree with identical predictions, but not an identical structure.

Thus:

A bottom-up tree is not necessarily identical to a top-down ID3 tree.

Reason: Greedy top-down selection is not globally optimal, so alternative construction orders can lead to legitimate but structurally different decision trees.

Question 6 — Optimization Geometry & Jensen's Inequality

1. Concavity of the Entropy Function

The binary entropy function is:

$$H(p) = -p \log_2 p - (1 - p) \log_2 (1 - p), \quad p \in (0, 1).$$

To analyze concavity, we compute the second derivative.

First derivative Rewrite using natural logarithms:

$$\log_2 p = \frac{\ln p}{\ln 2}.$$

Thus,

$$H(p) = -\frac{1}{\ln 2} [p \ln p + (1 - p) \ln(1 - p)].$$

Differentiate:

$$H'(p) = -\frac{1}{\ln 2} [\ln p + 1 - \ln(1 - p) - 1] = -\frac{1}{\ln 2} (\ln p - \ln(1 - p)).$$

Second derivative Differentiate again:

$$H''(p) = -\frac{1}{\ln 2} \left(\frac{1}{p} + \frac{1}{1-p} \right).$$

Since $p \in (0, 1)$, both denominators are positive. Therefore:

$$H''(p) = -\frac{1}{\ln 2} \left(\frac{1}{p(1-p)} \right) < 0.$$

$$\boxed{H(p) \text{ is strictly concave on } (0, 1).}$$

2. Jensen's Inequality and Non-Negativity of Information Gain

Information Gain measures the entropy reduction after splitting on a feature:

$$\text{Gain}(S, X) = H(Y) - \sum_v \frac{|S_v|}{|S|} H(Y_v),$$

where S_v is the subset of data where feature X takes value v .

Let the class probabilities in the subsets be p_v , and let:

$$\lambda_v = \frac{|S_v|}{|S|}.$$

We can write:

$$H(Y) = H \left(\sum_v \lambda_v p_v \right).$$

Because entropy is strictly concave (from part 1), Jensen's inequality gives:

$$\sum_v \lambda_v H(p_v) \leq H \left(\sum_v \lambda_v p_v \right) = H(Y).$$

Rearrange:

$$H(Y) - \sum_v \lambda_v H(p_v) \geq 0.$$

Thus:

$$\boxed{\text{Information Gain}(S, X) \geq 0.}$$

No feature split can *increase* entropy on average.

3. Optimization Trap: Hill-Climbing Failure in ID3

ID3 uses a greedy, top-down, hill-climbing strategy:

- At each node, choose the split with the **largest immediate Information Gain**.
- No backtracking or lookahead.

This can fail when the globally optimal tree requires a sequence of splits where the **first** split does *not* provide the highest gain.

Example scenario Suppose feature X_1 yields:

- A small immediate gain at the root,
- But after splitting on X_1 , the remaining features perfectly classify all samples.

Suppose feature X_2 yields:

- A large gain at the root,
- But leads to impure leaves that cannot be further split (or require deep, messy splits).

ID3 will pick X_2 because its immediate gain is higher. However, the best overall tree (minimal depth, perfect accuracy) is obtained by splitting on X_1 first.

This is a **local optimum**: the greedy choice looks best at the root but blocks access to the best global structure.

ID3 can get stuck because it optimizes short-term gain rather than exploring multi-step improvements.

Question 7 — The Failure of Greedy Search (XOR)

1. The Greedy Trap: Information Gain for X_1 and X_2

The XOR dataset is:

X_1	X_2	Y
0	0	0
0	1	1
1	0	1
1	1	0

Root entropy Two zeros and two ones:

$$H(Y) = 1.$$

Conditional entropy for X_1 When $X_1 = 0$: labels are $\{0, 1\}$ (entropy = 1). When $X_1 = 1$: labels are $\{1, 0\}$ (entropy = 1).

Weighted:

$$H(Y|X_1) = \frac{2}{4}(1) + \frac{2}{4}(1) = 1.$$

Thus:

$$\boxed{\text{Gain}(S, X_1) = 1 - 1 = 0.}$$

Conditional entropy for X_2 Identical symmetry:

- When $X_2 = 0$: labels $\{0, 1\} \rightarrow$ entropy 1 - When $X_2 = 1$: labels $\{1, 0\} \rightarrow$ entropy 1

So:

$$H(Y|X_2) = 1, \quad \boxed{\text{Gain}(S, X_2) = 0.}$$

Why ID3 fails ID3 selects the feature with the highest Information Gain. Here:

$$\text{Gain}(X_1) = \text{Gain}(X_2) = 0.$$

Thus the algorithm:

- has no preference at the root,
- may stop prematurely (no reduction in entropy),
- or chooses randomly,
- or produces a leaf predicting the majority label.

Hence ID3 *fails to even begin constructing* the XOR tree.

2. A Depth-2 Tree that Solves XOR

A correct XOR classifier must test both features. One valid tree:

$$\begin{aligned} \text{Root: } X_1 &\rightarrow \begin{cases} X_2 = 0 \Rightarrow Y = 0, \\ X_2 = 1 \Rightarrow Y = 1, \end{cases} & \text{if } X_1 = 0 \\ &\rightarrow \begin{cases} X_2 = 0 \Rightarrow Y = 1, \\ X_2 = 1 \Rightarrow Y = 0, \end{cases} & \text{if } X_1 = 1 \end{aligned}$$

This tree has:

$$\text{Depth} = 2, \quad \text{Training error} = 0.$$

3. Inductive Bias: Preference vs. Restriction

(a) Definition of Preference Bias

A **preference bias** does *not* limit the hypothesis space. Instead, it gives *priority* to certain hypotheses over others, typically the “simplest” ones according to some measure.

ID3 does not forbid any decision tree (full expressive power remains). However, it prefers:

small trees with high Information Gain near the root.

(b) Why XOR exposes ID3’s bias

In XOR:

- No feature yields positive Information Gain at the root. - A correct solution exists (depth-2 tree), but requires a split that appears useless *initially*. - ID3’s greedy criterion prevents it from exploring this path.

Thus ID3 does not fail because the hypothesis space is restricted, but because it has a **preference**:

ID3 prefers trees with large immediate Information Gain and therefore ignores globally optimal trees requiring low-gain root splits.

This is exactly why XOR—whose optimal tree requires two successive splits—exposes the greedy, local nature of the ID3 search process.

Question 8 — Diabetes Classification Using Support Vector Machines

Part 1: Data Preprocessing and Exploration

The Pima Indians Diabetes dataset contains 768 samples and 8 clinical predictors. Five medical features (Glucose, BloodPressure, BMI, SkinThickness, Insulin) contain invalid zero values. These were replaced using median imputation. The dataset was then split into 70% training and 30% testing, followed by standard feature scaling.

Why feature scaling is important. SVMs rely on geometric distances in feature space. Features with large numerical scales dominate the margin, leading to distorted hyperplanes. Scaling ensures equal contribution from all features and improves the optimization landscape.

Part 2: Linear SVM Classification

A linear SVM was trained both before and after normalization. The resulting accuracies were:

$$\text{Accuracy}_{\text{unscaled}} = 0.7229, \quad \text{Accuracy}_{\text{scaled}} = 0.7273.$$

The scaled model performs marginally better, consistent with SVM theory.

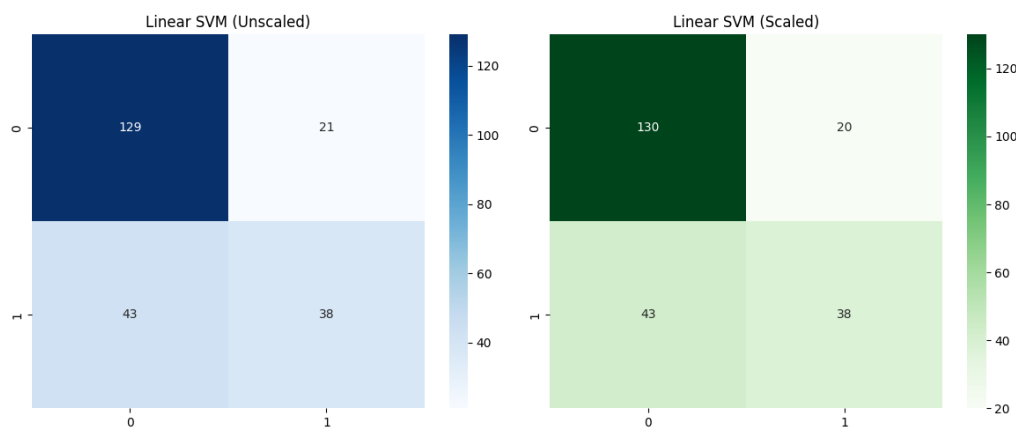


Figure 1: Confusion matrices for Linear SVM: unscaled (left) and scaled (right).

Effect of normalization. Normalization improves the conditioning of the optimization problem, leading to a more stable separating hyperplane.

Part 3: Comparison of Linear, RBF, and Polynomial Kernels

Three SVM classifiers were trained on the scaled data. The evaluation metrics are:

Kernel	Accuracy	Precision	Recall	F1
Linear	0.7273	0.6552	0.4691	0.5468
RBF	0.7403	0.6721	0.5062	0.5775
Polynomial	0.7273	0.7143	0.3704	0.4878

The RBF kernel achieves the highest accuracy and balanced performance across all metrics. The polynomial kernel exhibits high variance and limited recall, while the linear kernel underfits the nonlinear structure of the data.

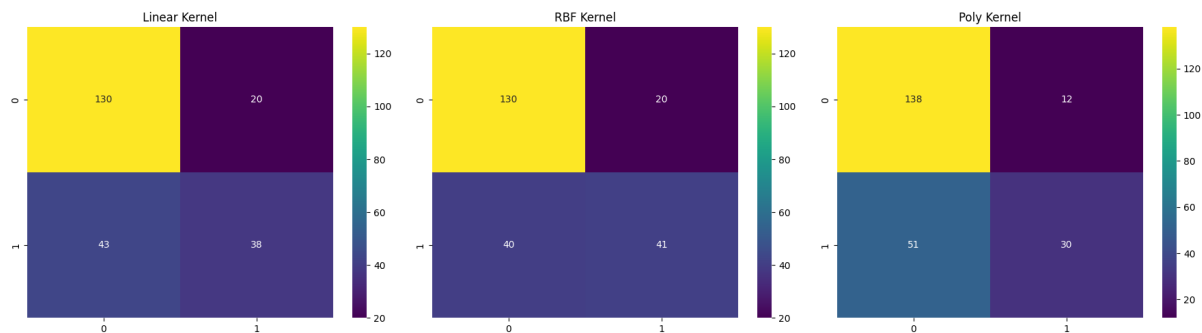


Figure 2: Confusion matrices for Linear, RBF, and Polynomial SVMs.

Part 4: Hyperparameter Tuning

GridSearchCV was applied to the RBF kernel with:

$$C \in \{0.1, 1, 10, 100\}, \quad \gamma \in \{0.001, 0.01, 0.1, 1\}.$$

The optimal parameters were:

$$C^* = 100, \quad \gamma^* = 0.001,$$

with cross-validated accuracy:

$$\text{Accuracy}_{\text{best}} = 0.7857.$$

Effect of large C . Very large C values heavily penalize misclassification, yielding tight, irregular decision boundaries and potential overfitting.

Part 5: PCA and 2D Visualization

PCA was applied to reduce the data from 8 to 2 dimensions. The projected training points are shown below.

Training an RBF SVM on the PCA space yields the decision boundary:

Effect on accuracy. PCA reduces discriminative information because it preserves variance rather than class separation. Thus, classification performance typically degrades after dimensionality reduction.

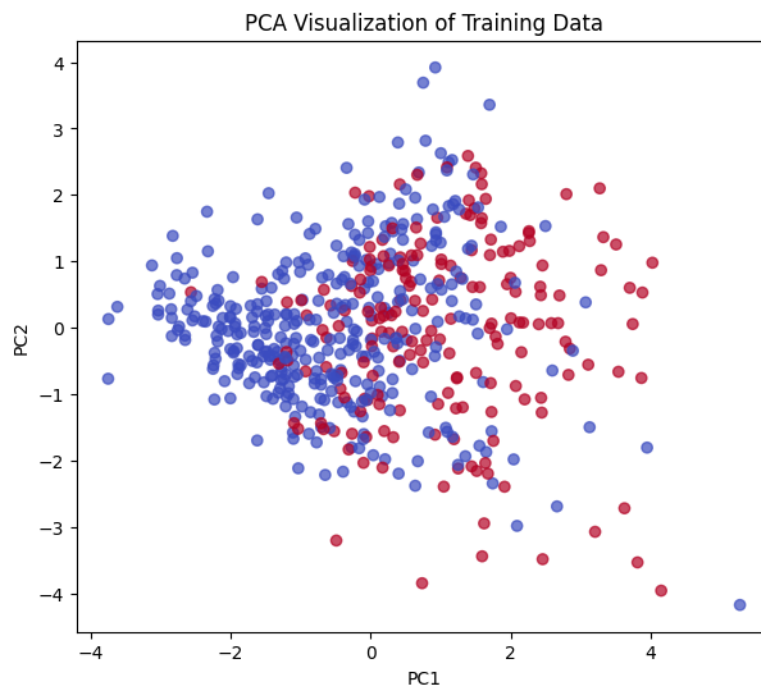


Figure 3: Training set after PCA reduction to 2D.

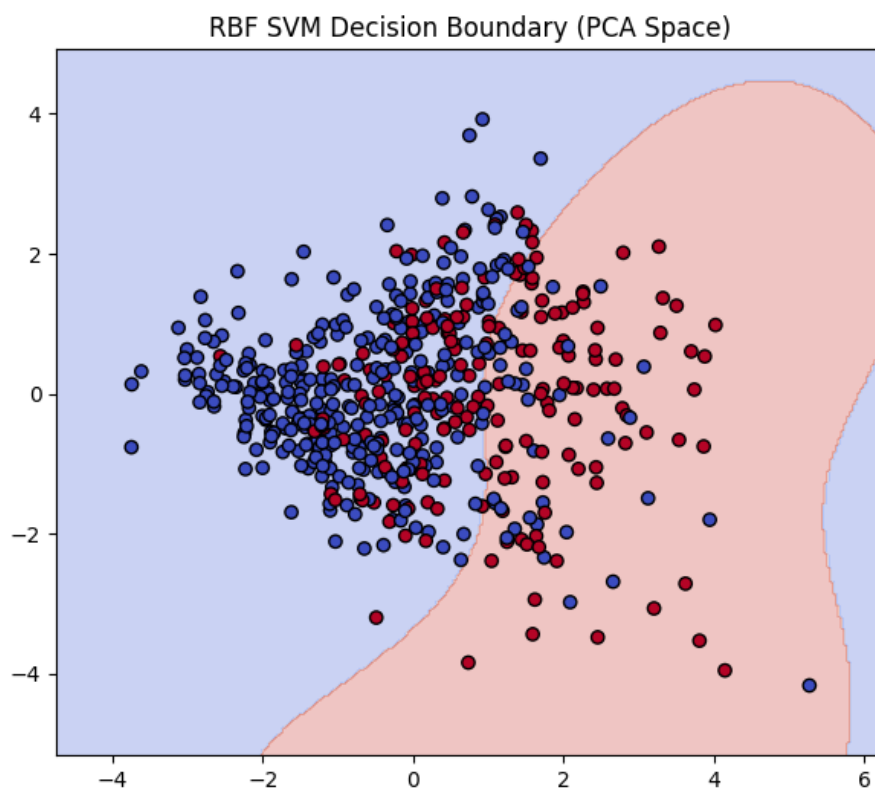


Figure 4: RBF SVM decision boundary in the PCA-reduced space.

Part 6: Class Imbalance Analysis

The dataset contains:

Class 0 : 500 samples, Class 1 : 268 samples.

An SVM trained with natural class weights yields:

$$\text{Recall}_{\text{standard}} = 0.5062.$$

Training with class balancing:

$$\text{Recall}_{\text{balanced}} = 0.6914.$$

Interpretation. Class balancing substantially improves recall for the minority diabetic class, increasing the clinical usefulness of the classifier by detecting more positive cases.

Question 9 — Primal Soft-Margin SVM via Quadratic Programming

Part 1: Data Preprocessing

The Social_Network_Ads dataset was used with two features (Age, EstimatedSalary). The target labels $\{0, 1\}$ were mapped to $\{-1, +1\}$. After a 70–30 train–test split, feature scaling (StandardScaler) was applied using only the training statistics to prevent data leakage.

Part 2: Solving the Primal SVM Using Quadratic Programming

We solve the primal soft-margin SVM:

$$\min_{w, b, \xi} \quad \frac{1}{2} \|w\|^2 + C \sum_{i=1}^n \xi_i$$

subject to

$$y_i(w^\top x_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0.$$

Using `cvxpy` with $C = 1$, the optimization produced:

$$w^* = \begin{bmatrix} 1.6821 \\ 1.0152 \end{bmatrix}, \quad b^* = -0.9117.$$

These parameters define the separating hyperplane:

$$w_1x_1 + w_2x_2 + b = 0.$$

Part 3: Model Evaluation

Predictions were computed using:

$$\hat{y} = \text{sign}(w^* \cdot x + b^*).$$

The final test accuracy of the QP-SVM model was:

$$\boxed{\text{Accuracy} = 0.8333}$$

which indicates strong performance given the simplicity of the linear model.

Part 4: Decision Boundary Visualization

The decision boundary in the 2D scaled feature space is shown below. The line is plotted using:

$$x_2 = -\frac{w_1}{w_2}x_1 - \frac{b^*}{w_2}.$$

The visualization shows a clean linear separation, demonstrating that the primal SVM solution recovers a margin-maximizing hyperplane directly from optimization principles, without using any high-level SVM API.

Question 10 — Decision Tree Classifier (ID3) from Scratch

Part 1: Data Loading and Pre-processing

The Extended Jeeves “Play Tennis” dataset was loaded using `pandas`. The feature `Day_ID` contains a unique identifier for each row, producing the following Information Gain:

$$IG(\text{Day_ID}) = 0.98145.$$

This occurs because each row forms a subset of size one, leading to zero entropy for every branch:

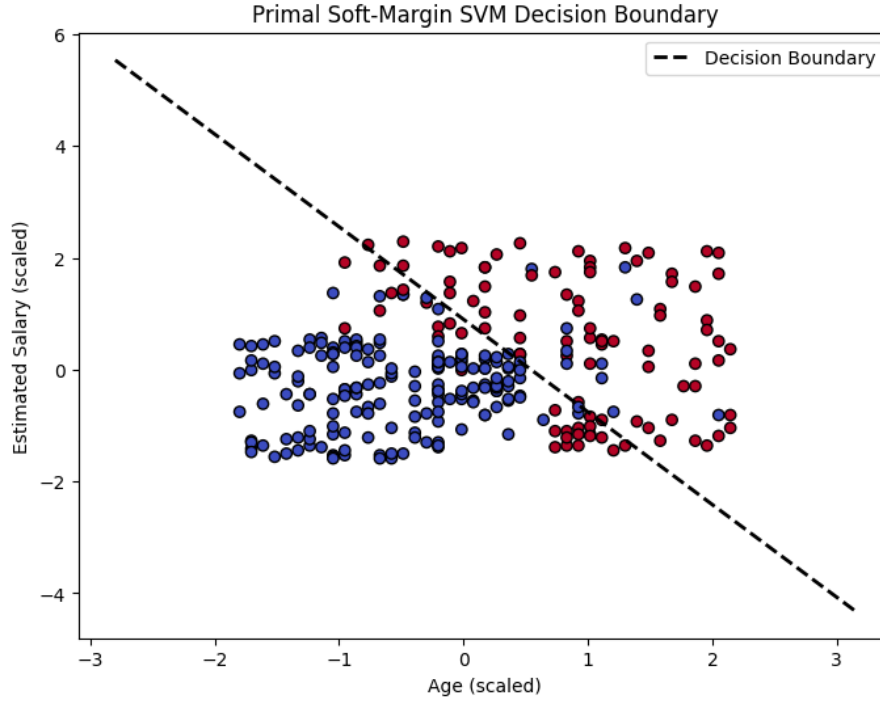


Figure 5: QP-based soft-margin SVM decision boundary using optimal parameters w^* and b^* .

$$IG = H(S) - \sum_{i=1}^n \frac{1}{n} H(\{y_i\}) = H(S).$$

Despite its mathematically maximal gain, `Day_ID` has no predictive value for unseen data and was therefore removed prior to tree construction.

Part 2: Entropy and Information Gain Computation

Entropy and Information Gain were implemented manually without using any machine learning library. Running these functions on the remaining discrete features yielded:

$$IG(\text{Outlook}) = 0.34307, \quad IG(\text{Humidity}) = 0.05389, \quad IG(\text{Wind}) = 0.03823.$$

Therefore, the best feature at the root of the decision tree is:

Outlook

Part 3: Handling Continuous Features (Temperature)

Continuous attributes were handled by evaluating all possible split points between adjacent samples with different target labels. For each candidate threshold, the expected entropy was computed, and the best split was selected. Applying this procedure to **Temperature** produced:

$$\text{Optimal threshold} = 30.05, \quad IG = 0.02392.$$

Thus the continuous split takes the form:

$$\text{Temperature} \leq 30.05 \quad \text{vs.} \quad \text{Temperature} > 30.05.$$

Part 4: Decision Tree Construction and Evaluation

A complete ID3 algorithm supporting both discrete and continuous features was implemented. After splitting the dataset into 80% training and 20% testing, the classifier achieved:

$$\text{Accuracy} = 0.94, \quad \text{Precision} = 0.9483, \quad \text{Recall} = 0.9483.$$

This demonstrates that the manually implemented decision tree successfully captures the dataset's structure.

Part 5: Overfitting, Noise Sensitivity, and Pruning

The full tree trained on clean data reached:

$$\text{Depth} = 42, \quad \text{Leaf nodes} = 14.$$

Due to the dataset's small size, the tree grows very deep while maintaining few leaf nodes.

After introducing 10% label noise, the learned tree expanded dramatically:

$$\text{Depth}_{\text{noisy}} = 13, \quad \text{Leaf nodes}_{\text{noisy}} = 101.$$

This behavior reflects the tendency of ID3 to memorize noisy data by creating a highly branched structure.

Pruning Experiment. Reduced Error Pruning was applied using the test set as validation data. At each pruning iteration, a subtree was replaced by a leaf whenever the validation accuracy did not decrease by more than a tolerance. The figure below plots the number of leaf nodes against the resulting test accuracy.

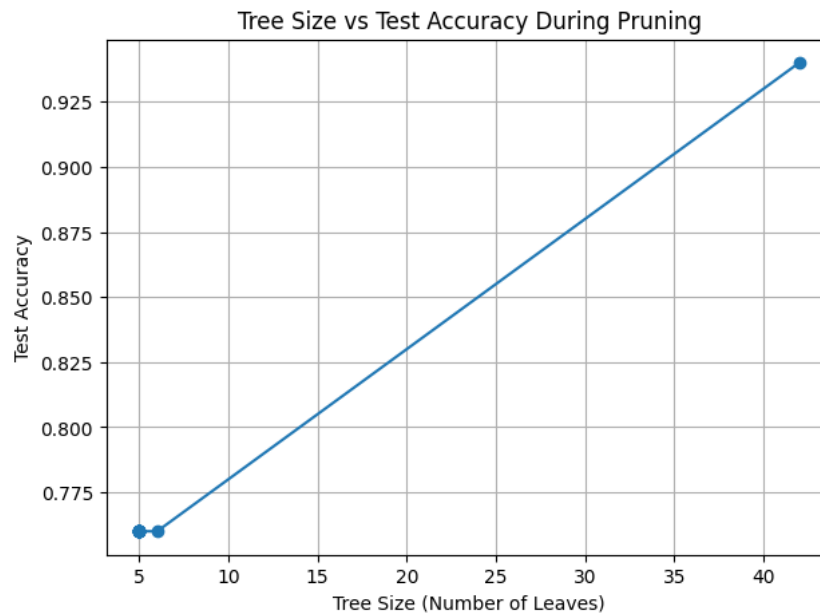


Figure 6: Tree Size vs. Test Accuracy during Reduced Error Pruning.

Generalization Analysis. The pruning curve reveals the classical trade-off between complexity and generalization. Very small trees underfit and achieve low accuracy (around 0.76). As tree size increases, accuracy improves steadily. The largest tree (≈ 42 leaves) attains the best accuracy (≈ 0.94), indicating that the dataset is simple and highly structured, and that pruning removes essential information. In this case, reduced pruning yields the best generalization performance.